

Predicting Used Car Prices: An End-to-End MLOps pipeline

Diogo Tibério, 20250341
José Montez, 20250351
Henrique Figueiredo, 20250433
Sebastião Jerónimo, 20240660

Table of contents

- 1. Introduction and Dataset Selection 1
- 2. Project Planning..... 1
- 3. Pipeline Architecture 2
 - 3.1 Medallion Architecture 2
 - 3.2. Data Exploration 2
 - 3.3. Data Quality 2
 - 3.4. Feature Engineering and Feature Store 3
 - 3.5. Model Selection 4
 - 3.6 End-to-End Workflows 4
- 4. Data Drift 5
- 5. Model Deployment..... 6
- 6. Packages and Versions 6
- 7. Annex..... 7

1. Introduction and Dataset Selection

Cars 4 You is an online marketplace for used vehicles that currently relies on a network of mechanics to inspect each car and estimate its market value before listing it for sale. As the platform has grown, this manual evaluation process has become an operational bottleneck, limiting scalability and increasing the time required to publish new listings.

To address this challenge, the company aims to automate the vehicle valuation process by developing a machine learning model capable of estimating a car's selling price from information provided by the seller. The goal is to enable the company to process a larger volume of listings with lower operational costs.

This project uses a used-car dataset containing approximately 76,000 training examples, where each vehicle is described by 13 attributes, including brand, model, year, mileage, fuel type, transmission, engine size, and previous ownership information. The objective is to build an end-to-end MLOps pipeline that performs data validation, feature engineering, model training, evaluation, and prediction in a reproducible and modular manner using Kedro.

Model performance is evaluated using the regression metrics MAE, RMSE, and R^2 , with RMSE selected as the primary optimization metric. This choice reflects the business requirement of minimizing large pricing errors, as substantial over- or underestimation of a vehicle's value has a greater financial and reputational impact than many smaller prediction errors.

2. Project Planning

The project was developed following an iterative approach inspired by Agile methodologies, where the implementation was divided into five successive tasks. Each task focused on a specific stage of the MLOps workflow (Table 1) and produced a functional, testable component before progressing to the next stage. This approach provides independent development and validation of each stage before integrating them into the complete MLOps workflow.

Table 1- Overview of project tasks, components and corresponding Kedro pipelines.

Task	Focus	Components	Kedro Pipeline
1	Data Quality	Great Expectations suite, unit tests on raw data. Data and schema validation, missing values, invalid ranges and categorical consistency checks	data_quality
2	Feature engineering and Feature store	Data standardization, correction of inconsistencies, handling invalid values, and preparation of clean datasets	data_cleaning
3	Data Preparation	Training dataset split, feature engineering, preprocessing for training and batch inference	split_pipeline, preprocess_train, preprocess_batch
4	Model Development & Inference	Model training, evaluation, and batch prediction using the trained model.	model_train, model_predict

3. Pipeline Architecture

The project was implemented using Kedro in order to achieve a modular, reproducible, and maintainable machine learning workflow. Each processing stage is encapsulated in an independent pipeline with explicit data dependencies, allowing individual components to be developed, tested, and executed separately or as part of an end-to-end workflow.

The resulting architecture is organized into two end-to-end workflows: a training pipeline, responsible for building and evaluating new models from labelled historical data, and an inference pipeline, which processes new, unseen vehicle listings to generate price predictions. Both workflows reuse the same data cleaning and preprocessing logic, ensuring consistency between training and production inference

3.1 Medallion Architecture

The data processing workflow follows a Medallion Architecture, which organizes data into successive quality layers that separate raw, cleaned, and processed data. This layered approach provides clear data lineage, allowing each dataset to be traced back to its source and the transformations applied throughout the pipeline. It also promotes separation of concerns, with each layer responsible for a specific stage of the data lifecycle, while improving the reproducibility and maintainability of the overall workflow.

The **Bronze** layer contains the raw input data together with the results of the data quality validation. The **Silver** layer stores the cleaned and standardized datasets produced by the data cleaning pipeline. Finally, the **Gold** layer contains the feature-engineered datasets, trained models, evaluation metrics, and batch predictions generated by the training and inference workflows.

3.2. Data Exploration

Exploratory analysis was conducted exclusively on the training set to avoid information leakage from the test batch. The target variable price is right-skewed, with a mean of £16,882 and a median of £14,699, ranging from £450 to £159,999, the upper tail being driven by a small number of luxury and prestige vehicles. Ford is the most represented brand at approximately 20% of listings, with manual transmission and petrol as the dominant categorical values. Pairwise analysis of numeric features revealed that engine size, year, and mileage are the variables most strongly associated with price, in the expected directions: larger engines and newer cars command a premium while higher mileage depresses value. Regarding categorical data exploration, there was an extended data inconsistency in the brand, car model, transmission and fuel type columns, with brand alone containing 72 distinct values despite representing a much smaller number of actual manufacturers, and similar patterns were observed across the other features.

3.3. Data Quality

The data quality issues identified during EDA were formalized into an explicit validation contract using Great Expectations, applied as an automated quality gate at the

entry point of the pipeline. The suite defined expectations across all features, covering four categories: **completeness** (price non-null), **valid numeric ranges** (mileage, tax, mpg, engineSize, previousOwners non-negative; paintQuality% between 0-100; year between 1990-2026), **categorical membership** (fuelType and transmission restricted to known canonical values), and an **explicit flag on hasDamage as a constant-zero column**, capturing the suspicious pattern identified during exploration.

The same suite was applied to the incoming test batch, simulating how a production system would validate new data arriving from an upstream feed, with results stored as Great Expectations data documents for auditing purposes.

3.4. Feature Engineering and Feature Store

Five new features were derived from the cleaned data to capture relationships observed during exploration, primarily between price and the age, mileage, and ownership history of a vehicle (Table 2).

Table 2-Engineered features added during the feature engineering step.

Feature	Rationale
car_age	More directly interpretable measure of depreciation than the raw manufacture year.
mileage_per_year	Distinguishes a high-mileage car driven steadily over many years from one driven intensively over a short period.
is_high_mileage	Allows the model to capture a potential threshold effect rather than assuming a purely linear relationship.
is_low_paint_quality	Marks the point at which paint condition is likely to materially affect resale value.
owner_to_age_ratio	Captures how frequently a vehicle has changed hands relative to its age, a stronger depreciation signal than the raw ownership count.

The engineered dataset underwent a second Great Expectations validation pass, before being ingested into the Hopworks Feature Store. Each row was keyed by the original car id, with the ingestion timestamp recorded as the event time, and configured to automatically compute summary statistics and correlations for every feature. A Feature View was then created over this Feature Group, selecting all available features and designating price as the target label, establishing a single source of truth that decouples the feature engineering step from the model training step that follows.

It should be noted that the Hopworks Serverless free tier does not provide an offline storage connector, which prevented materializing the training dataset directly through `feature_view.get_training_data()`. As a workaround, the training data used in the following section was built from the same validated, feature-engineered DataFrame that was ingested into the Feature Group, recreating what the Feature View would return.

3.5. Model Selection

With features validated and stored in the Feature Store, the modelling stage focused on comparing multiple algorithms under a consistent experimental protocol, with every run tracked in MLflow to ensure full reproducibility and auditability. The dataset was split 70/15/15 into training, validation, and test sets. Categorical features were one-hot encoded for Linear Regression and label-encoded for the gradient-boosted models, consistent with each algorithm's handling of categorical inputs.

Three models were trained and compared. A linear regression model served as a performance floor that any more complex model would need to beat to justify its added cost. XGBoost and LightGBM were then tuned using Optuna. For each model family, 20 trials were run, searching over parameters such as the number of estimators, tree depth, learning rate, and regularization strength. Every trial was logged as a nested MLflow run under a parent run, producing a complete audit trail of the search process and allowing later inspection of which hyperparameter regions were most promising. The validation-set performance of the three final models is summarized below.

Table 3- Validation performance of the three candidate models.

Model	RMSE	MAE	R^2
XGBoost	£2,249	£1,275	0.9447
LightGBM	£2,282	£1,302	0.9430
Linear Regression	£4,491	£2,681	0.7795

XGBoost achieved the lowest RMSE and was selected as the Champion model, narrowly outperforming LightGBM and substantially outperforming the Linear Regression baseline. All three models were registered in the MLflow Model Registry, with the Champion and the two Challengers tagged accordingly, preserving the option to roll back or re-compare model versions in the future. The Champion was then evaluated on the held-out test set, which it had not seen during training or hyperparameter tuning, using MLflow's built-in evaluation framework (Table 4).

Table 4- Final test-set performance of the XGBoost Champion model.

Metric	Value
RMSE	£2,108
MAE	£1,363
R^2	0.9499
MAPE	8.67%
Max Error	£20,679

3.6 End-to-End Workflows

In the pipeline registry, the seven individual pipelines are registered both as standalone pipelines and as two composed workflows for production use (Annex Figure 1). Both *data_quality* and *data_cleaning* are shared as the first two steps in both workflows, meaning that regardless of whether the pipeline is processing labeled training data or an

unlabeled inference batch, the same validation contract and the same cleaning logic are applied before any other processing takes place.

The training workflow, after cleaning the data is partitioned into training and validation subsets by *split_train*. *preprocess_train* then fits all preprocessing transformations — missing value imputation, feature encoding, and scaling, exclusively on the training partition and persists all fitted artifacts to disk. New variables are also derived at this stage, after imputation, engineered training features are then pushed to the Hopsworks Feature Store as a dedicated feature group as mentioned in Section 3.4. The encoded and scaled feature are passed to *model_train* which trains the best configuration of each model as a final run and the logs all metrics and artifacts to MLFlow as well as the new Champion model.

The inference workflow receives an unlabeled batch of approximately 19,000 listings, in this case *preprocess_batch* loads the preprocessing artifacts persisted by the most recent training run and applies them directly to the cleaned batch data, without refitting any transformation. The Champion model is then loaded and predicts a price for every listing in the batch.

4. Data Drift

Data drift happens when new data starts to look different from the data a model was trained on. When this occurs, the model's predictions can become unreliable, so the goal of this component is to detect it automatically.

We used the Evidently library (v0.6.5) with its *DataDriftPreset*, which compares a *reference* dataset (the data the model was trained on) against a *current* dataset and checks, column by column, whether the distributions changed. Evidently selects an appropriate statistical test for each column automatically and reports overall dataset drift when more than 50% of the columns drift. As an extra check, we also computed the Population Stability Index (PSI) per feature, using the threshold from the practical classes (PSI > 0.2 indicates significant drift). To confirm the detector works we split the data into a reference half and a current half. For the reference half we didn't do any changes to the data, and as expected, no drift was detected by the evidently report.

For the current half we manually created drift on purpose by altering the engineered features. On this batch, the detector correctly fired, and the flagged columns matched exactly the columns that were changed - confirming the component is sensitive to real drift but does not false-alarm on normal data. We implemented this in two ways: an exploration notebook (which runs the baseline check and the manufactured-drift demo, saving both HTML reports) and a Kedro pipeline (*data_drifts*) that loads the model-input data, runs the drift check, and saves the report and metrics to *08_reporting*.

In a real system this check would run automatically on new data; if drift is detected, it would raise an alert and trigger retraining on fresh data. The main decision is the choice of threshold – if the threshold is too high it might miss real drift, if too low it may cause false alarms.

5. Model Deployment

The Champion model was containerized to ensure a consistent, reproducible runtime environment across development and production. The trained XGBoost model was wrapped in a FastAPI application exposing a REST endpoint for real-time price prediction, alongside a health-check endpoint reporting whether the service is up or down, then packaged into a Docker image to pin dependencies and eliminate environment drift.

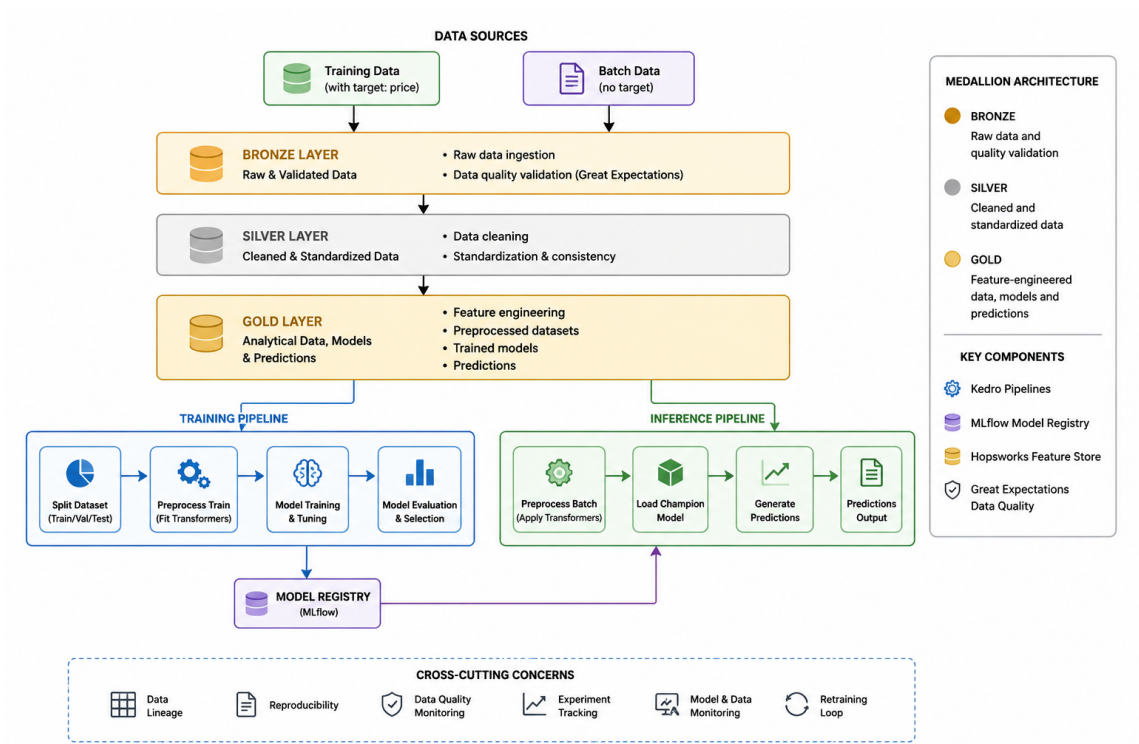
For orchestration, the containerized service was deployed to Kubernetes, which manages scheduling, scaling, and restarts of the prediction service. Its declarative model allows replicas to scale with request volume, while built-in health checks restart failed containers automatically. This also enables rolling updates, so a new Champion model can be deployed with gradual traffic shifting and an immediate rollback option.

This containerized, orchestrated design decouples the model-serving infrastructure from the data and training pipelines, so the prediction service can be scaled or updated independently of the Kedro pipelines that produce new model versions. As a proof of concept, the deployment was validated locally; a production rollout would additionally require a container registry, CI/CD integration to automate image builds on model promotion, and monitoring of latency and request volume alongside the data drift checks described above.

6. Packages and Versions

Package	Version
Python	3.13.1
Kedro	1.3.1
kedro-mlflow	2.0.2
great-expectations	1.18.1
hopsworks	3.x
mlflow	3.x
optuna	4.x
lightgbm	4.x
xgboost	2.x
scikit-learn	1.x
shap	0.46.x
fastapi	0.115.x
evidently	0.6.x
pandas	2.x
numpy	2.x
pytest	8.x
pytest-cov	6.x

7. Annex



Annex Figure 1- End-to-end training and inference workflows composed from the seven Kedro pipelines, following the Medallion Architecture from raw data ingestion (Bronze) through cleaning (Silver) to feature engineering, model training, and prediction (Gold). Both workflows share the data_quality and data_cleaning steps before diverging: the training pipeline fits preprocessing on the training split and trains the Champion model, while the inference pipeline reuses the persisted preprocessing artifacts to score new batch data.